

1's complement
complement every bit.

Eg: $(101110)_2$

→ 010001

2's complement

→ From left, copy all zeroes till first 1.

→ Copy first 1

→ Complement all remaining bits.

Eg: $(101100)_2 \rightarrow$ 010100

2's com. Binary Addition. Eg:

$0 + 0 \rightarrow 0$	Carry
$0 + 1 \rightarrow 1$	0
$1 + 0 \rightarrow 1$	0
$1 + 1 \rightarrow 0$	1

① 1 0 1
(+) 1 1 0 0

1 0 1 1 1

① ① ① ① ① ① ①
0 1 1 0 1 0 1 0

0 0 0 0 1 0 0 0

1 0 0 0 0 0 0 1

(+) 1 1 1 1 1 1 1 1

1 1 1 1 1 0 0 1 0

Binary Subtraction

$0 - 0 \rightarrow 0$	Borrow
$0 - 1 \rightarrow 1$	1
$1 - 0 \rightarrow 1$	0
$1 - 1 \rightarrow 0$	0

Eg: ①

1 0 1 1

(-) 0 1 1 0

0 1 0 1

Binary Multiplication

Eg:

1 0 0 1

x 1 1 0 1

① 1 0 0 1

0 0 0 0

1 0 0 1

(+) 1 0 0 1

1 1 1 0 1 0 1

Binary Division

Eg:

1 1 0 1

1001 | 1 1 1 0 1 0 1

(-) 1 0 0 1 ↓

0 1 0 1 1

(-) 1 0 0 1 ↓

0 0 1 0 0 1

(-) 1 0 0 1

0 0 0 0

decimal number
 decimal digits

Decimal	BCD			
	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

⇒ Decimal no. to BCD

(i) $(17)_{10} \rightarrow 0001\ 0111$ BCD

(ii) $(156)_{10} \rightarrow 0001\ 0101\ 0110$ BCD

⇒ BCD to Decimal

(i) $\underline{10100}$ BCD $\rightarrow (14)_{10}$

(ii) $\underline{1001001}$ BCD $\rightarrow (49)_{10}$

⇒ Comparison between binary and BCD..

	Binary	BCD
$(10)_{10} \rightarrow$	1010	00010000
$(12)_{10} \rightarrow$	1100	00010010

Excess-3 CODE: Add 3 (0011) to BCD.

Eg: Decimal - 35

BCD - 0011 0101

Excess 3 - 0110 1000

case 1: sum ≤ 9 with carry = 0

decimal	BCD
2	0010
+ 6	0110
8	01000 ← valid

$$(2)_{10} + (6)_{10} = (8)_{10}$$

case 3: sum > 9 , carry = 0

7	^⓪ 0111
+ 6	0110
13	^⓪ 1101 ← invalid
	+ 0110 Add 6
	00010011
	1 3

GRAY CODE: Reflected Binary code (RBC)

* Two succ. values differ in only 1 bit.

Decimal	Binary				Gray			
	b ₃	b ₂	b ₁	b ₀	g ₃	g ₂	g ₁	g ₀
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

XOR gate

BCD Addition

Sum ≤ 9
Carry = 0
(no correction required)

Sum ≥ 9
Carry = 1
(Add 6 to the sum)

Sum > 9
Carry = 0
(Add 6 to the sum)

case 2: sum ≥ 9 with carry = 1

9	1001
+ 8	1000
17	110001 ← Add 6 to invalid BCD
	+ 0110
	00010111
	1 7

⇒ Add (83)₁₀ and (34)₁₀ in BCD

83	1000 0011
+ 34	0011 0100
117	^⓪ 1011 0111
	→ invalid BCD
	+ 0110

000100010111
1 1 7

⇒ Binary to Gray code

$$g_3 = b_3$$

$$g_2 = b_2 \oplus b_3$$

$$g_1 = b_1 \oplus b_2$$

$$g_0 = b_0 \oplus b_1$$

Eg: b₃ b₂ b₁ b₀

1	0	1	1
⊕	⊕	⊕	⊕
↓	↓	↓	↓
1	1	1	0
g ₃	g ₂	g ₁	g ₀

⇒ Gray code to Binary

Eg:

$$b_3 = g_3$$

$$b_2 = g_2 \oplus g_3$$

$$b_1 = g_1 \oplus g_2$$

$$b_0 = g_0 \oplus g_1$$

1	0	0	1
⊕	⊕	⊕	⊕
↓	↓	↓	↓
1	1	1	0

There is a wide variety of binary codes used in digital systems. Some of these codes are binary- coded-decimal (BCD), Excess-3, Gray, octal, hexadecimal, etc. Often it is required to convert from one code to another. For example the input to a digital system may be in natural BCD and output may be 7-segment LEDs. The digital system used may be capable of processing the data in straight binary format. Therefore, the data has to be converted from one type of code to another type for different purpose. The various code converters can be designed using gates.

1) Binary Code:

It is straight binary code. The binary number system (with base 2) represents values using two symbols, typically 0 and 1. Computers call these bits as either off (0) or on (1). The binary code are made up of only zeros and ones, and used in computers to stand for letters and digits. It is used to represent numbers using natural or straight binary form.

It is a weighted code since a weight is assigned to every position. Various arithmetic operations can be performed in this form. Binary code is weighted and sequential code.

2) Gray Code:

It is a modified binary code in which a decimal number is represented in binary form in such a way that each Gray- Code number differs from the preceding and the succeeding number by a single bit. Whereas by using binary code there is a possibility of change of all bits if we move from one number to other in sequence (e.g. binary code for 7 is 0111 and for 8 it is 1000). Therefore it is more useful to use Gray code in some applications than binary code.

The Gray code is a non weighted code i.e. there are no specific weights assigned to the bit positions. Like binary numbers, the Gray code can have any no. of bits. It is also known as reflected code.

Applications:

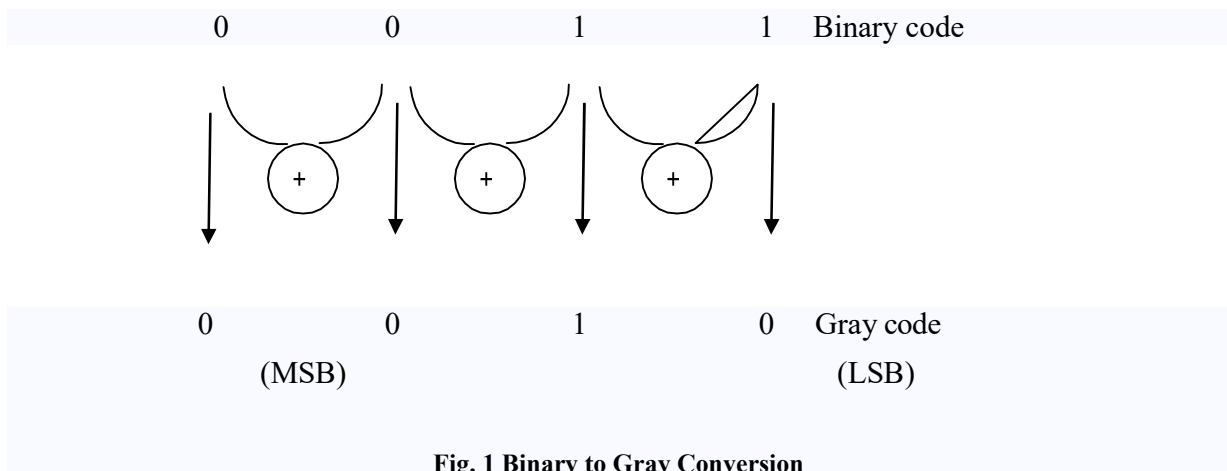
1. Important feature of Gray code is it exhibits only a single bit change from one code word to the next in sequence.
2. It is sometimes convenient to use the Gray code to represent the digital data converted from the analog data (Outputs of ADC).
3. Gray codes are used in angle-measuring devices in preference to straight forward binary encoding.
4. Gray codes are widely used in K-map

The disadvantage of Gray code is that it is not good for arithmetic operation

Binary to Gray Conversion

In this conversion, the input straight binary number can easily be converted to its Gray code equivalent.

1. Record the most significant bit as it is.
2. EX-OR this bit to the next position bit, record the resultant bit.
3. Record successive EX-ORed bits until completed.
4. Convert 0011 binary to Gray.



Gray to Binary Conversion

1. The Gray code can be converted to binary by a reverse process.
2. Record the most significant bit as it is.
3. EX-OR binary MSB to the next bit of Gray code and record the resultant bit.
4. Continue the process until the LSB is recorded.
5. Convert 1011 Gray to Binary code.

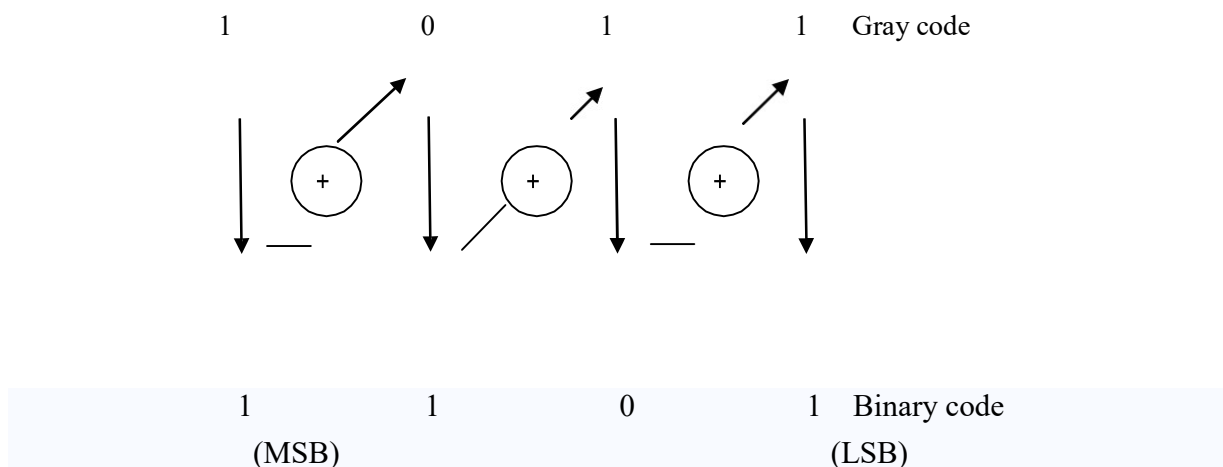


Fig. 2 Gray to Binary Conversion

3) BCD Code:

Binary Coded Decimal (BCD) is used to represent each of decimal digits (0 to 9) with a 4-bit binary code. For example $(23)_{10}$ is represented by 0010 0011 using BCD code rather than $(10111)_2$. This code is also known as 8-4-2-1 code as 8421 indicates the binary weights of four bits ($2^3, 2^2, 2^1, 2^0$). It is easy to convert between BCD code numbers and the familiar decimal numbers. It is the main advantage of this code. With four bits, sixteen numbers (0000 to 1111) can be represented, but in BCD code only 10 of these are used. The six code combinations (1010 to 1111) are not used and are invalid.

Applications: Some early computers processed BCD numbers. Arithmetic operations can be performed using this code. Input to a digital system may be in natural BCD and output may be 7-segment LEDs.

It is observed that more number of bits are required to code a decimal number using BCD code than using the straight binary code. However in spite of this disadvantage it is very convenient and useful code for input and output operations in digital systems.

4) EXCESS-3 Code:

Excess-3, also called XS3, is a non weighted code used to express decimal numbers. It can be used for the representation of multi-digit decimal numbers as can BCD. The code for each decimal number is obtained by adding decimal 3 and then converting it to a 4-bit binary number. For e.g. decimal 2 is coded as $0010 + 0011 = 0101$ in Excess-3 code.

This is self complementing code which means 1's complement of the coded number yields 9's complement of the number itself. Self complementing property of this helps considerably in performing subtraction operation in digital systems, so this code is used for certain arithmetic operations.

BCD To Excess – 3 Code Conversions:

Convert BCD 2 i. e. 0010 to Excess – 3 codes

For converting 4 bit BCD code to Excess – 3, add 0011 i. e. decimal 3 to the respective code using rules

of binary addition.

$$0010 + 0011 = 0101 - \text{Excess} - 3 \text{ code for BCD } 2$$

Excess – 3 Code To BCD Conversion:

The 4 bit Excess-3 coded digit can be converted into BCD code by subtracting decimal value 3 i.e. 0011 from 4 bit Excess-3 digit.

e.g. Convert 4-bit Excess-3 value 0101 to equivalent BCD code.

$$0101 - 0011 = 0010 - \text{BCD for } 2$$

Design:

A) Binary to Gray Code Conversion:

1) Truth Table:

Table 1 Binary to Gray Code Conversion

INPUT (BINARY CODE)				OUTPUT (GRAY CODE)			
B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

2) K-Map for Reduced Boolean Expressions of Each Output:

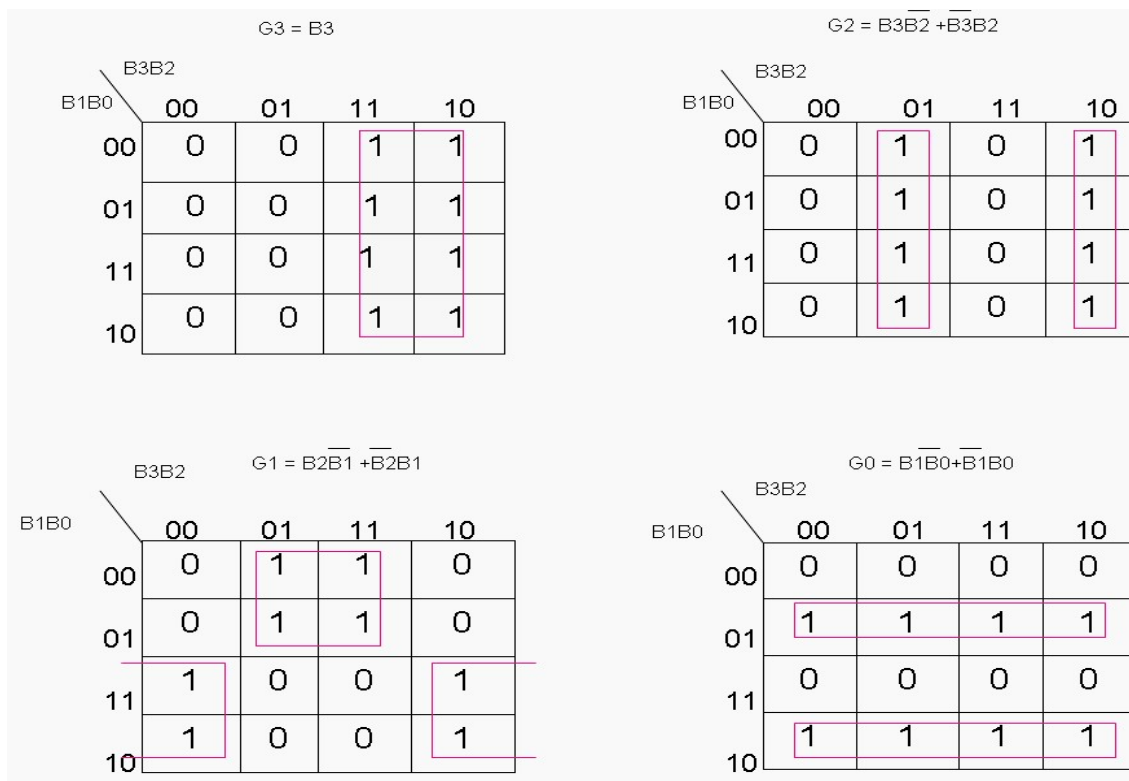


Fig. 4 K-Map for Reduced Boolean Expressions of Each Output (Gray Code)

3) Circuit Diagram:

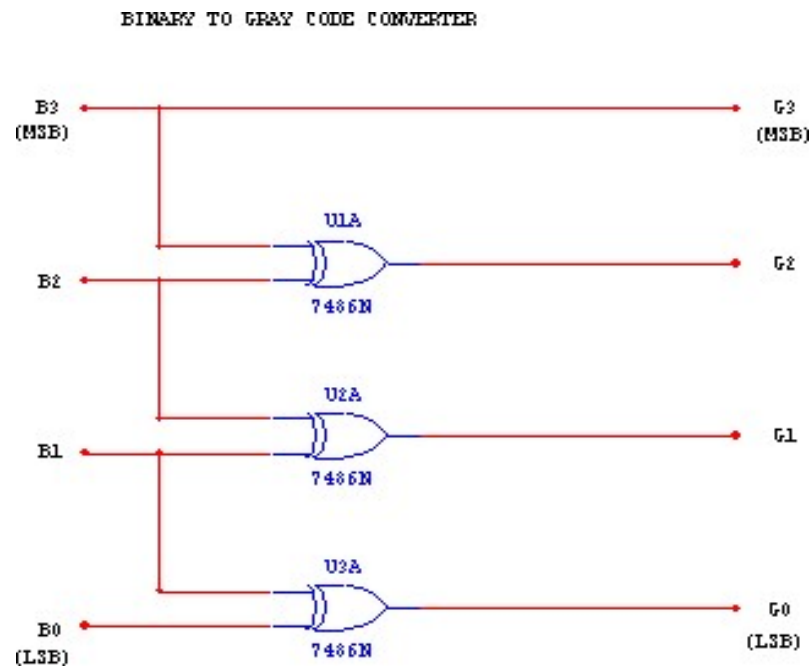


Fig. 5 Logical Circuit Diagram for Binary to Gray Code Conversion

B) Gray to Binary Code Conversion:

1) Truth Table:

Table 2 Gray to Binary Code Conversion

INPUT (GRAY CODE)				OUTPUT (BINARY CODE)			
G ₃	G ₂	G ₁	G ₀	B ₃	B ₂	B ₁	B ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1

0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

2) K-Map for Reduced Boolean Expressions of Each Output:

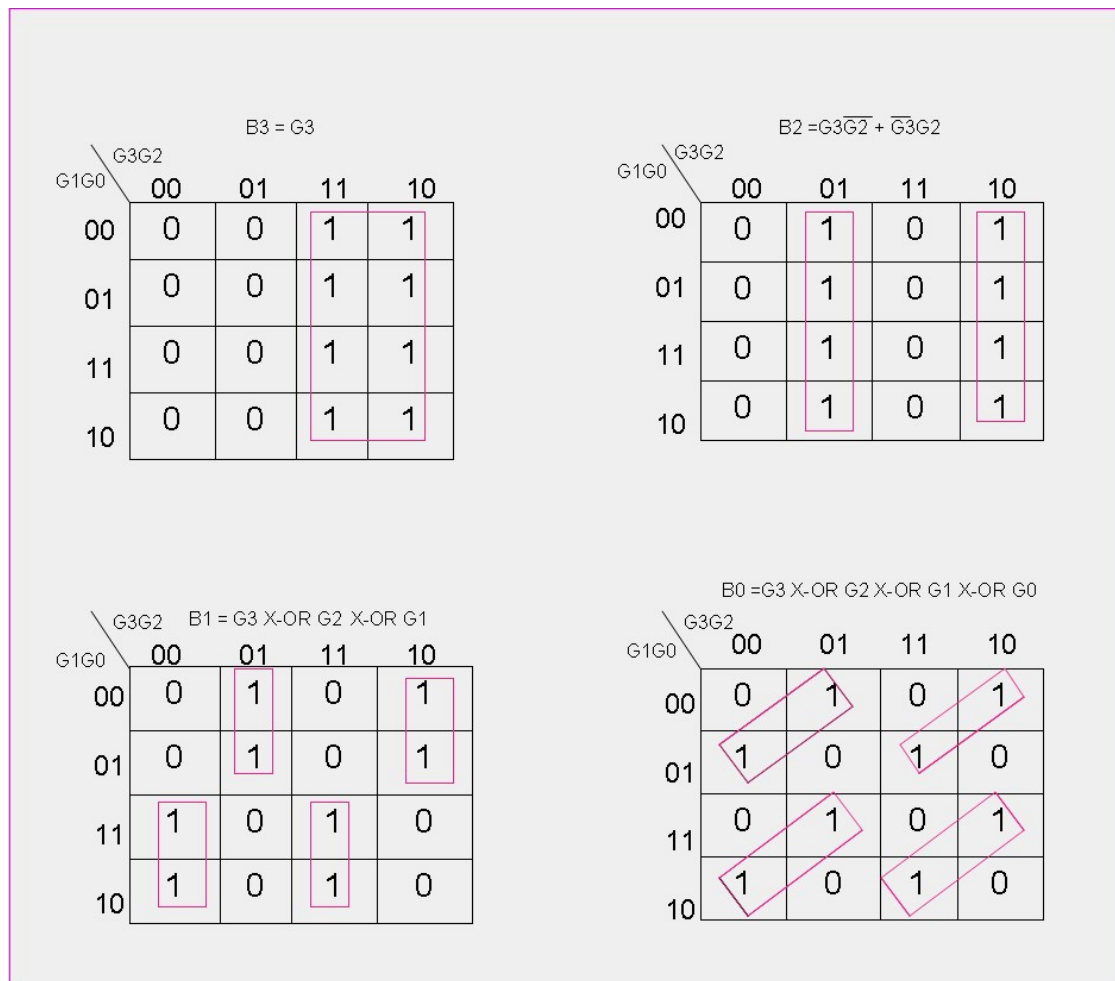
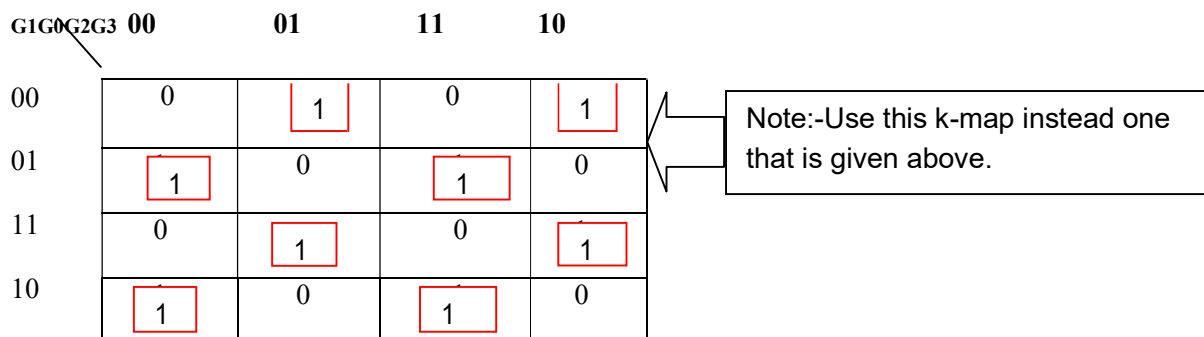


Fig. 6 K-Map for Reduced Boolean Expressions of Each Output (Binary Code)



$$B0 = G3 \text{ X-OR } G2 \text{ X-OR } G1 \text{ X-OR } G0$$

3) Circuit Diagram:

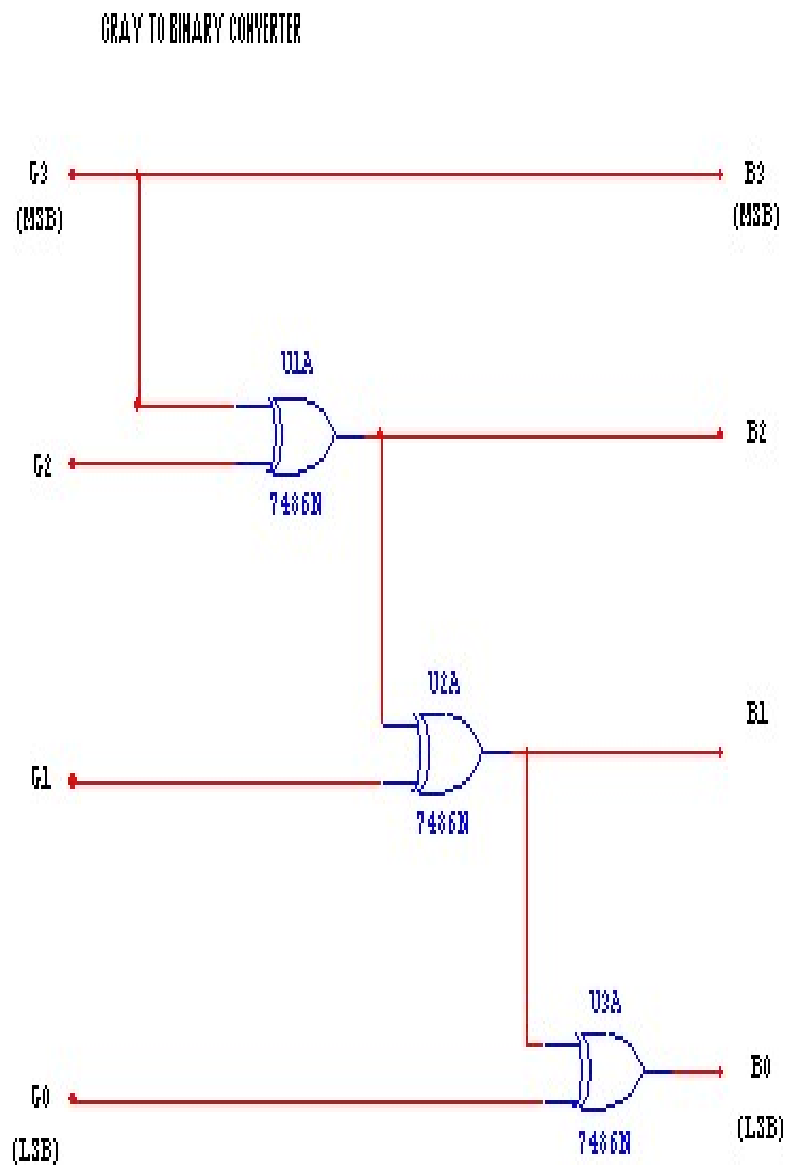


Fig. 7
Logical Circuit Diagram for Gray to Binary Code Conversion

C) BCD to Excess-3 Code Conversion:

1) Truth Table:

Table 3 BCD to Excess-3 Code Conversion

INPUT (BCD CODE)				OUTPUT (EXCESS-3 CODE)			
B ₃	B ₂	B ₁	B ₀	E ₃	E ₂	E ₁	E ₀
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	x	x	x
1	0	1	1	X	x	x	x
1	1	0	0	X	x	x	x
1	1	0	1	X	x	x	x
1	1	1	0	X	x	x	x
1	1	1	1	X	x	x	x

2) K-Map for Reduced Boolean Expressions of Each Output:

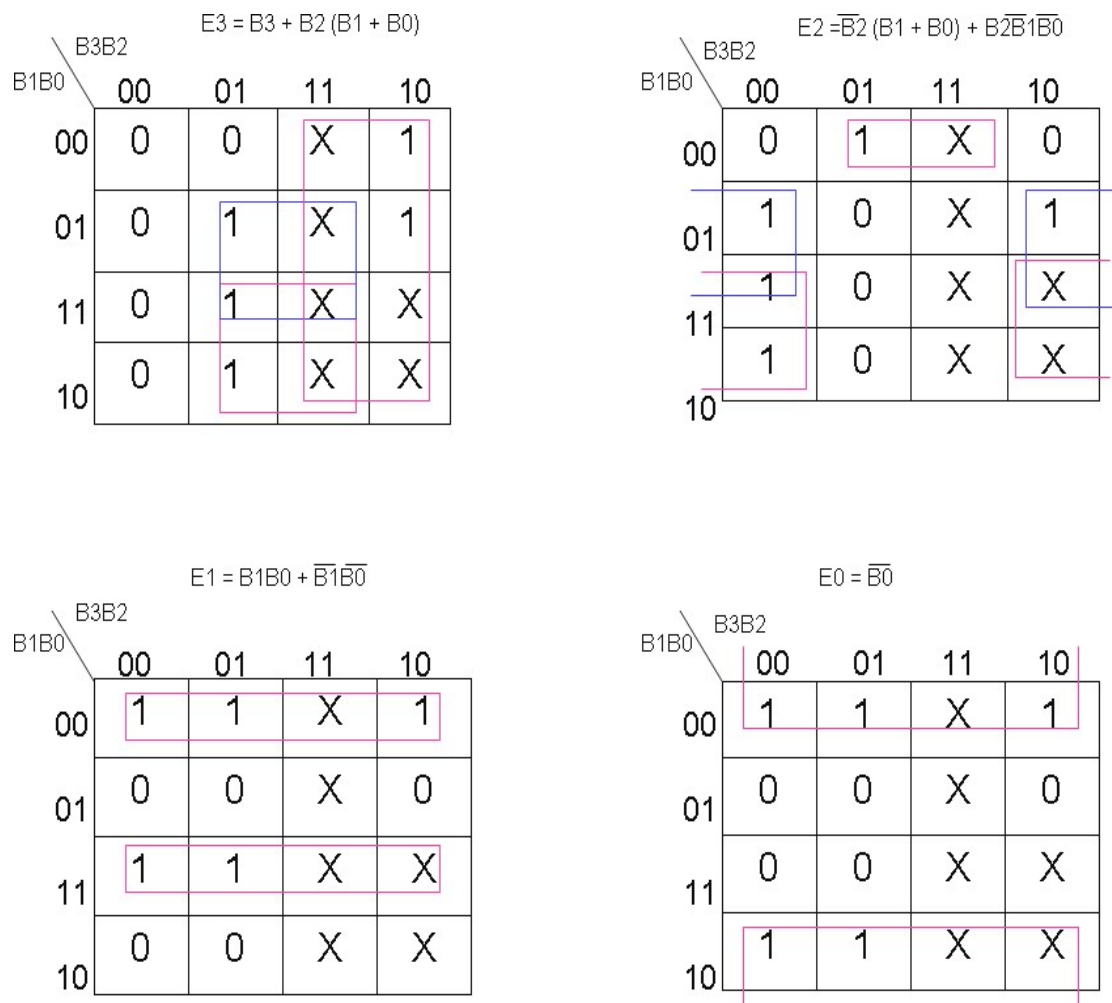


Fig. 8 K-Map for Reduced Boolean Expressions Of Each Output (Excess-3 Code)

3) Circuit Diagram:

BCD TO EXCESS-3 CONVERTER

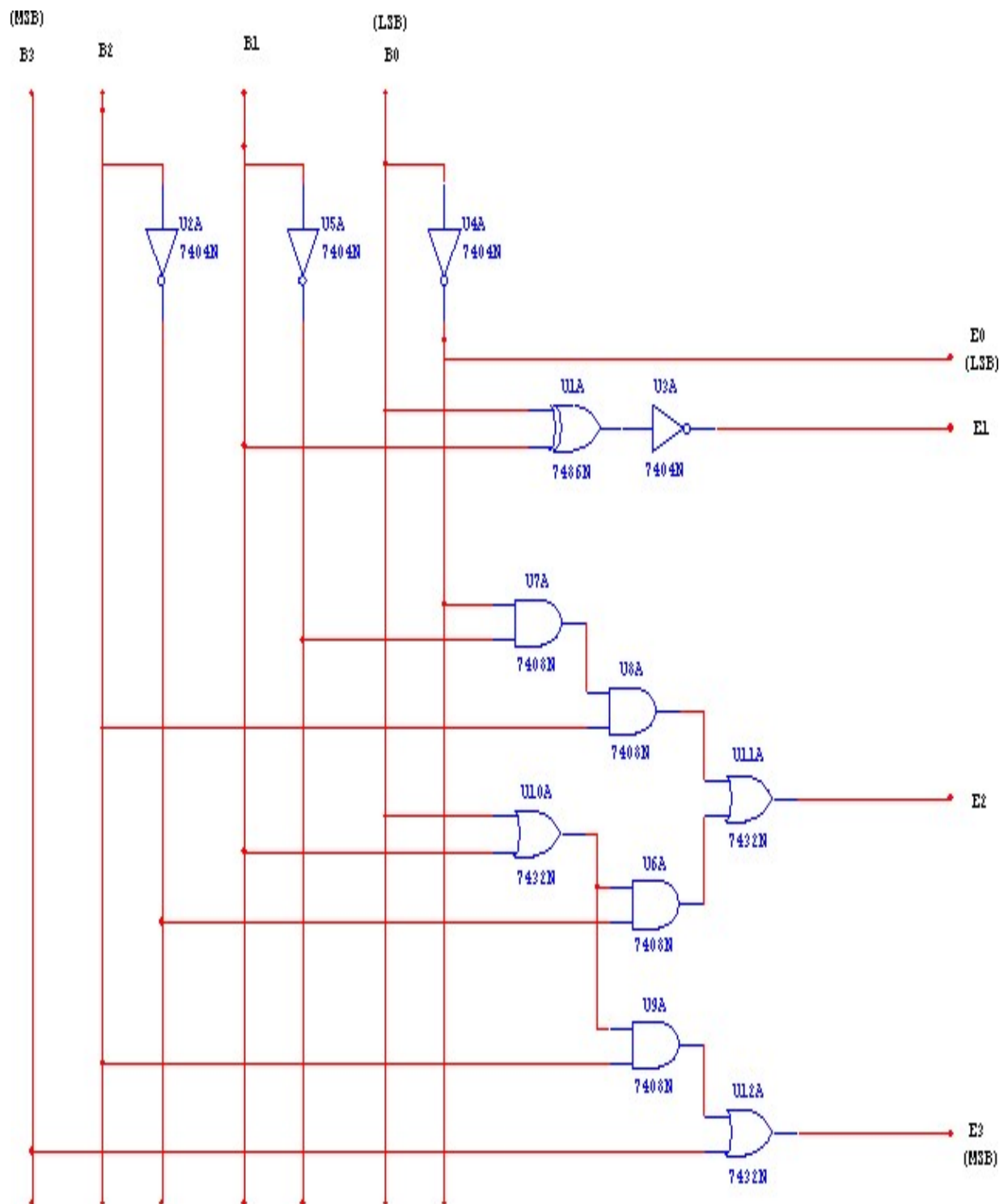


Fig.9 Logical Circuit Diagram for BCD to Excess-3 Code Conversion

D) Excess-3 to BCD Conversion:

1) Truth Table:

Table 4 Excess-3 To BCD Conversion

INPUT (EXCESS-3 CODE)				OUTPUT (BCD CODE)			
E ₃	E ₂	E ₁	E ₀	B ₃	B ₂	B ₁	B ₀
0	0	0	0	X	X	X	X
0	0	0	1	X	X	X	X
0	0	1	0	X	X	X	X
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

2) K-Map for Reduced Boolean Expressions of Each Output:

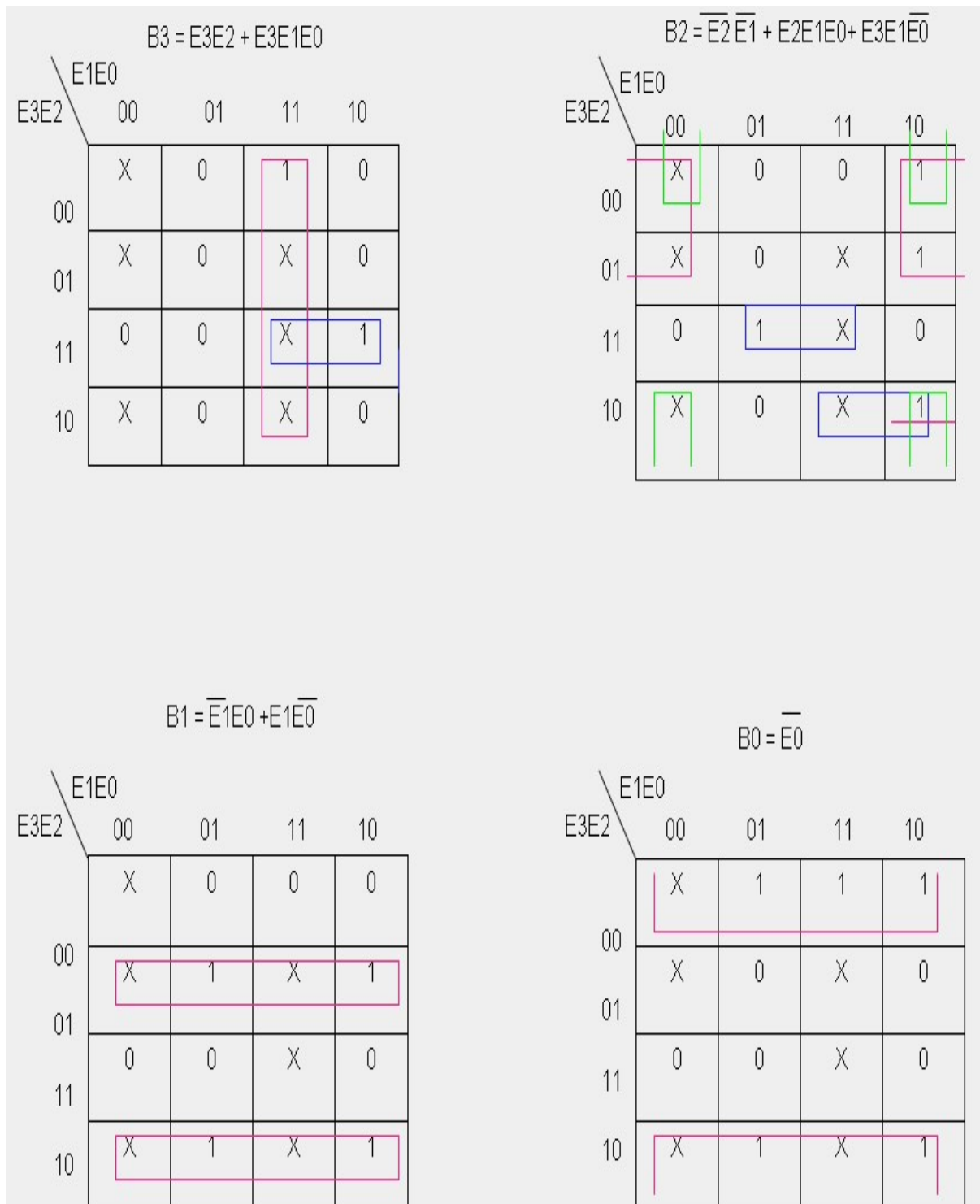


Fig 10 K-Map For Reduced Boolean Expressions of Each Output (BCD Code)

3) Circuit Diagram:

EXCESS-3 TO BCD CONVERTER

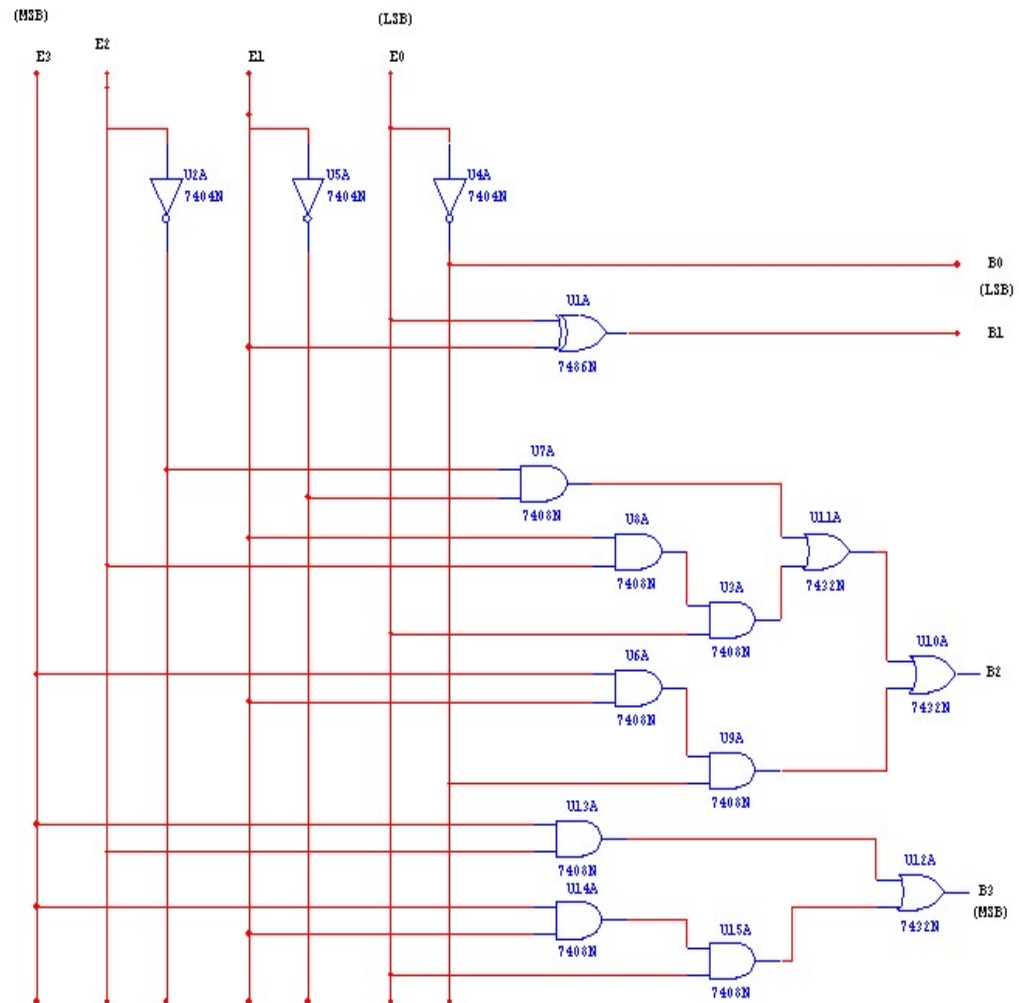


Fig.11 Logical Circuit Diagram for Excess-3 to BCD Conversion

Outcome:

Thus, we studied different codes and their conversions including applications.

The truth tables have been verified using IC 7486, 7432, 7408, and 7404.